Machine Learning II

Aula 9 — Transformers

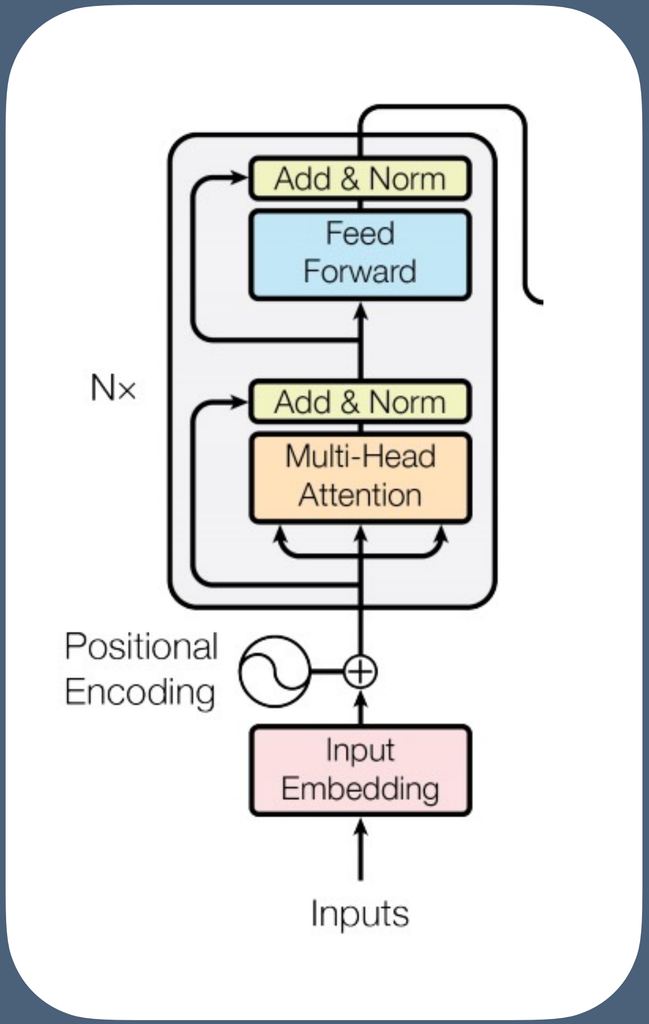
Daniel Yukimura, Francisco Ganacim
IMPA

15 de abril de 2026

Transformers

Conceitos principais:

- **Tokens**
- **Atenção**
- **Encode posicional**



Mais uma arquitetura?

Imagens	→	**CNN**
Sequências	→	**RNN**
Grafos	→	**GNN**

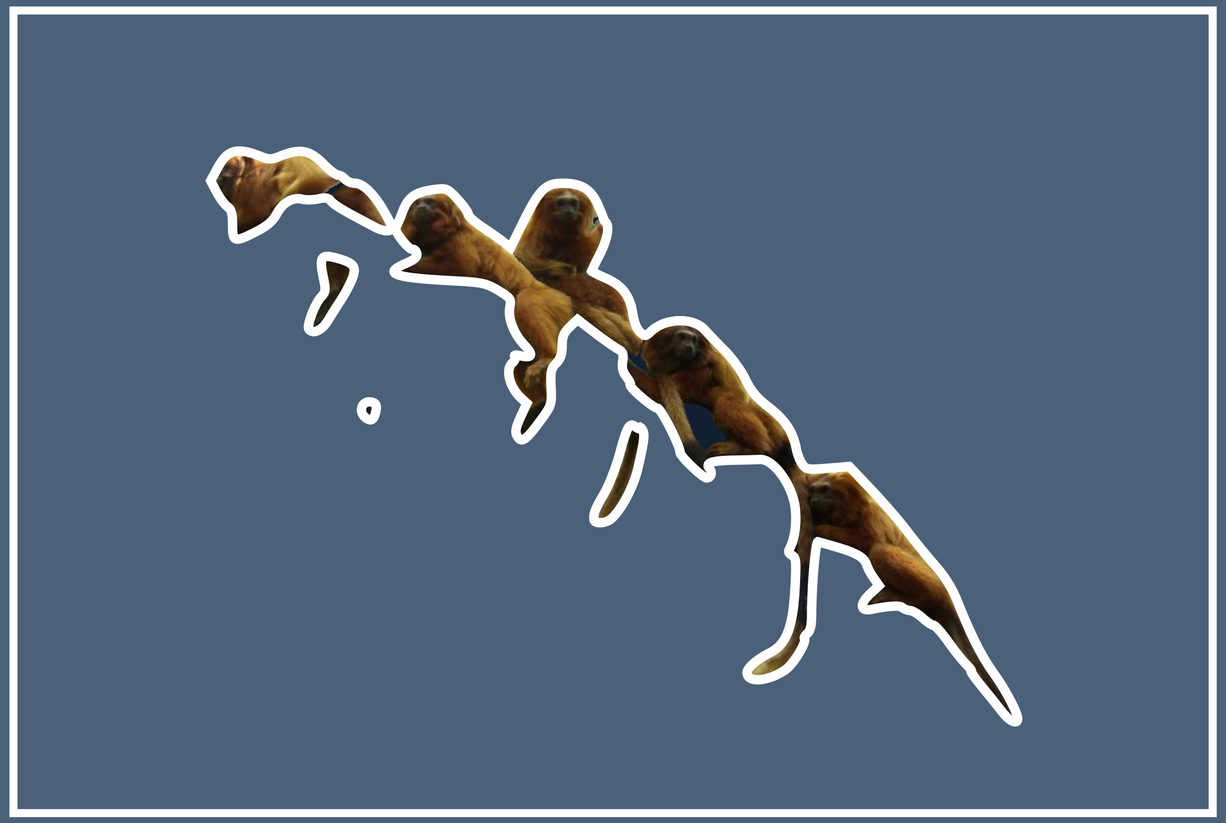
Limitações da CNN

- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?

- CNNs são especializadas para operações locais, e não funcionam tão bem para relações de longa distância.

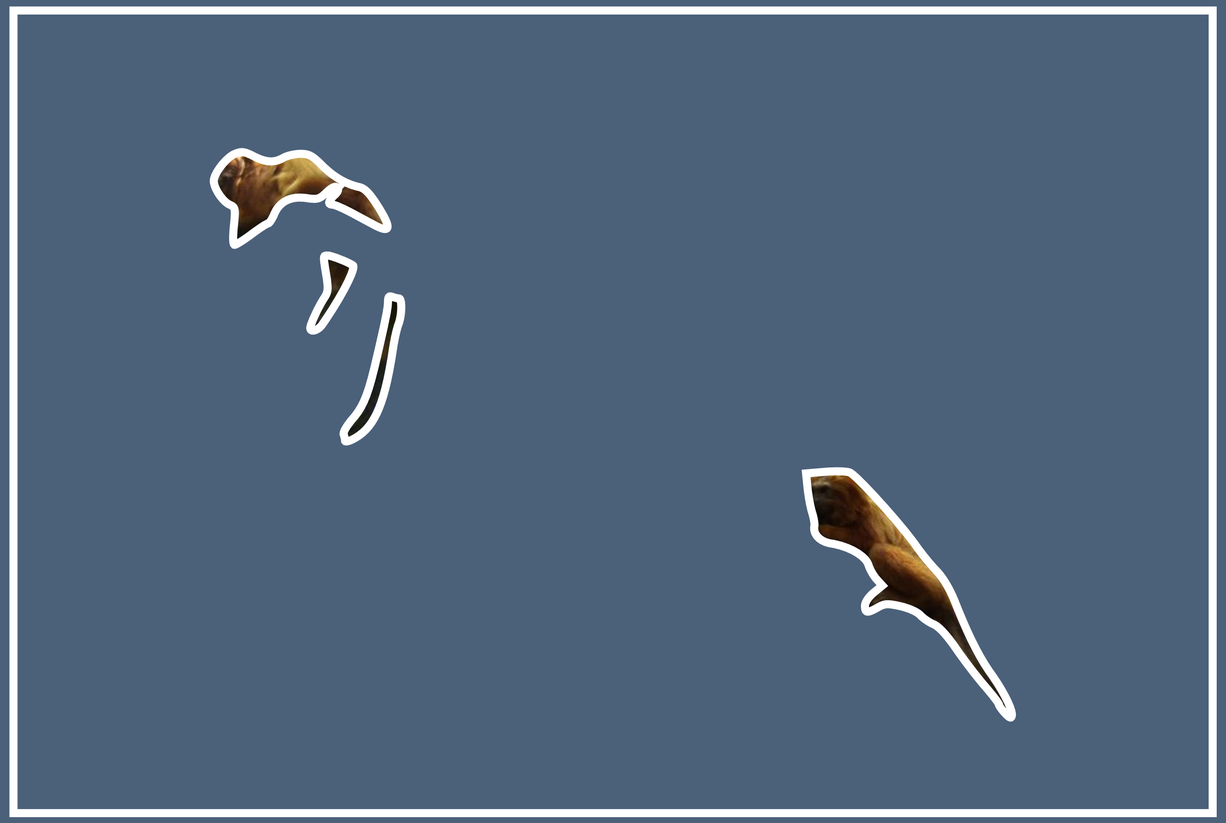
A ideia de atenção

A ideia de atenção



- Quantos micos tem nessa imagem?

A ideia de atenção

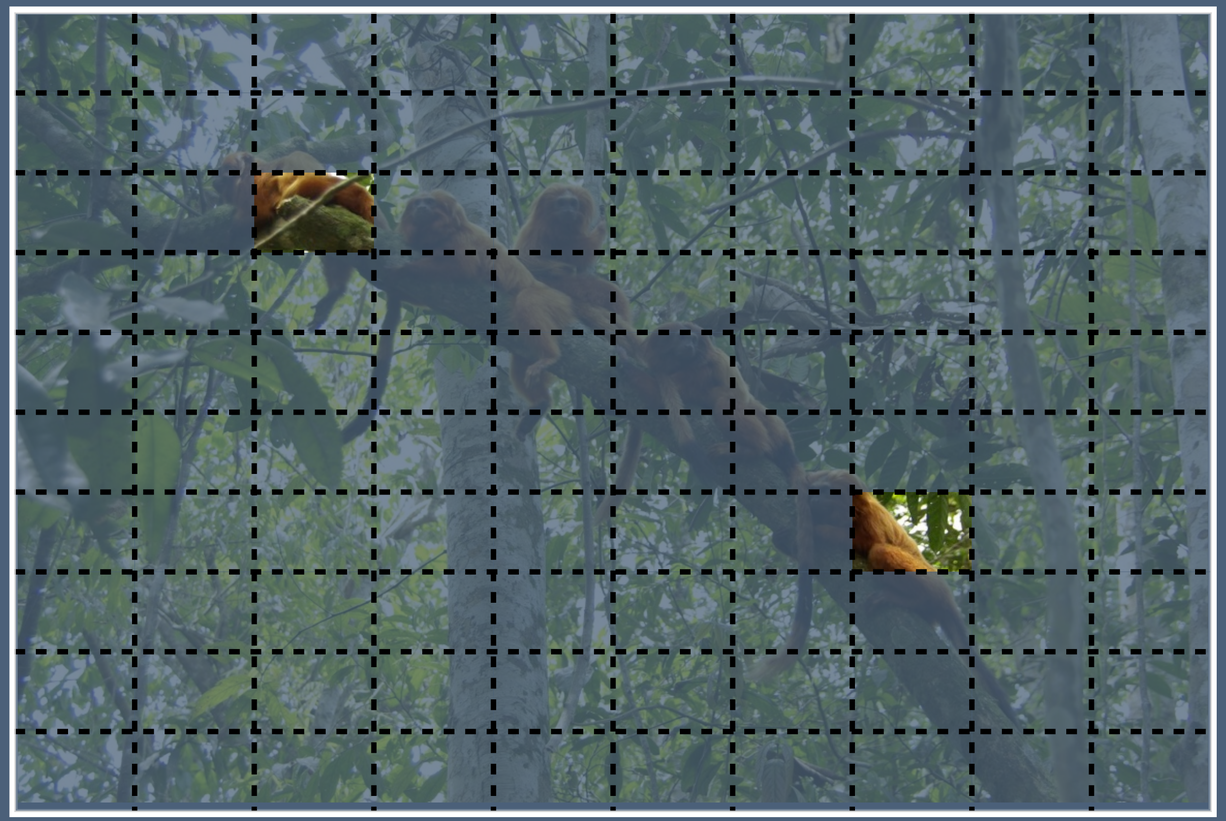


- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?

A ideia de atenção

- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?

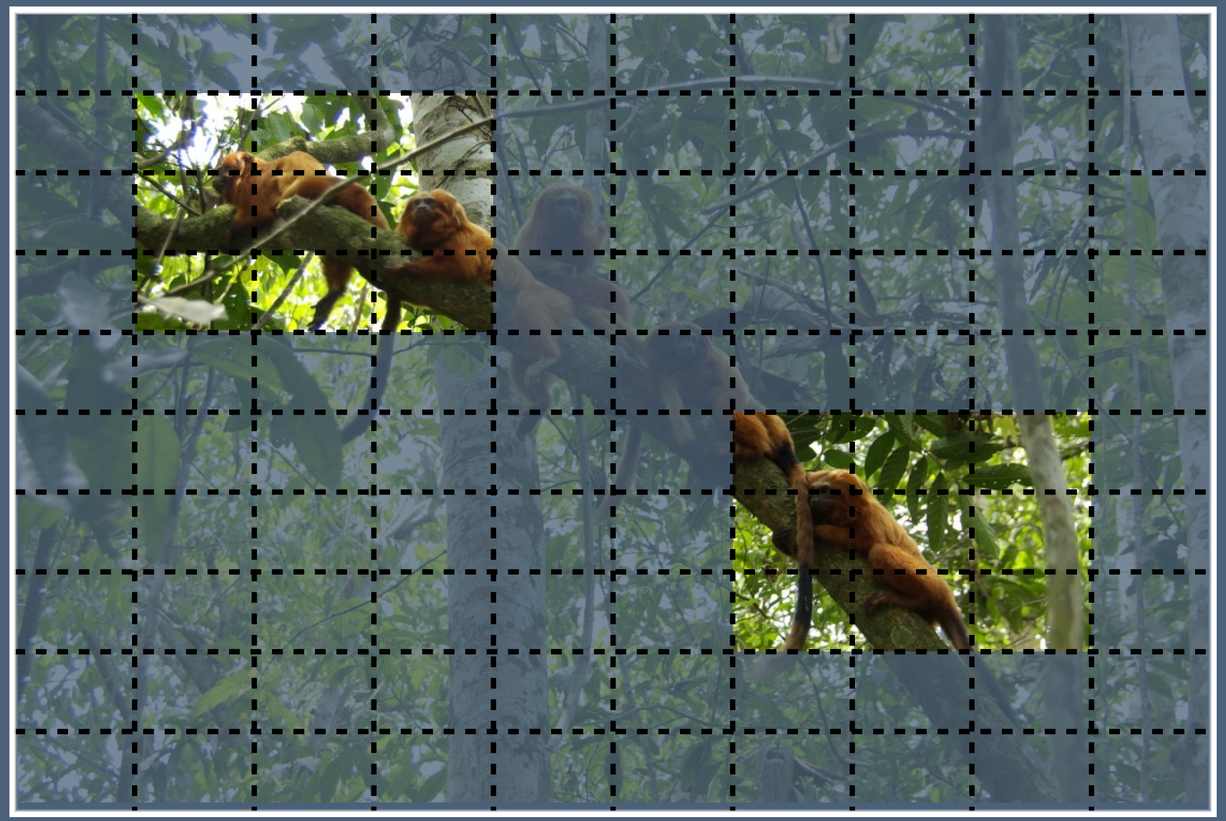
A ideia de atenção



- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?

- CNNs são especializadas para operações locais, e não funcionam tão bem para relações de longa distância.

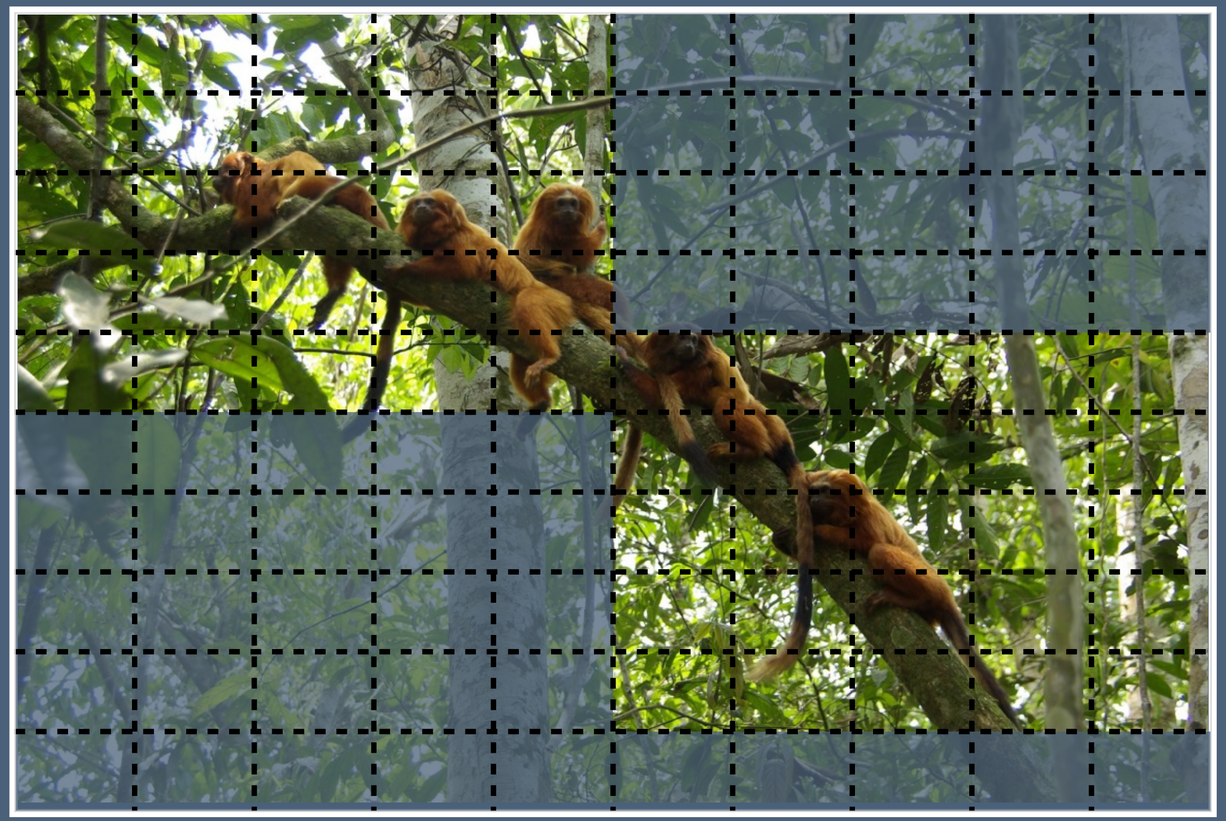
A ideia de atenção



- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?

- CNNs são especializadas para operações locais, e não funcionam tão bem para relações de longa distância.

A ideia de atenção

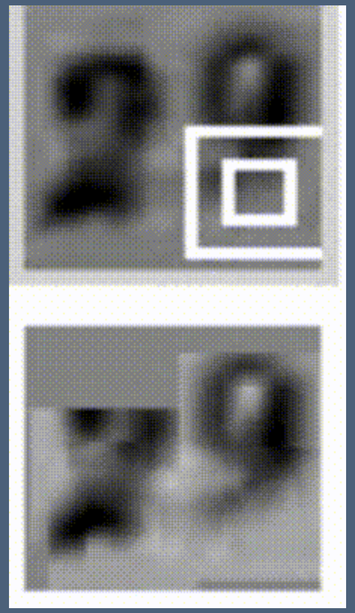


- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?

- CNNs são especializadas para operações locais, e não funcionam tão bem para relações de longa distância.

A ideia de atenção

- Quantos micos tem nessa imagem?
- O macaco mais acima na árvore é da mesma espécie do que está mais abaixo?



Multiple Object Recognition with Visual Attention — Ba et al 2014

Entendendo Transformers

1. **Tokens**
2. **Atenção**
3. **Encode posicional**

Tokens

Exemplo: Como prever a próxima palavra

Essa manhã eu levei meu cachorro para passear

considerando essas palavras

prever

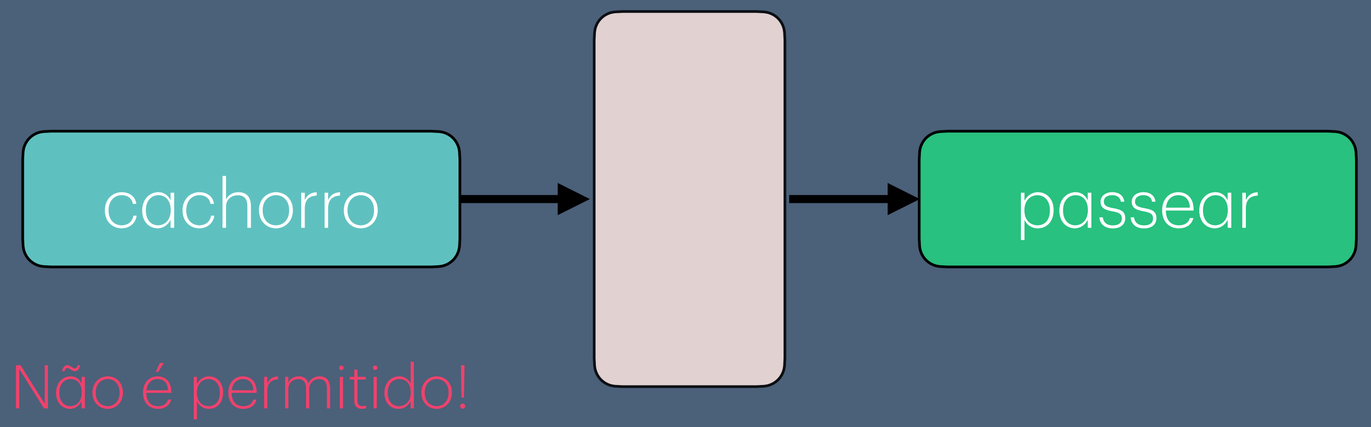
Como prever a próxima palavra?

Essa manhã eu levei meu cachorro para passear

considerando essas palavras

prever

Representando palavras para Redes Neurais



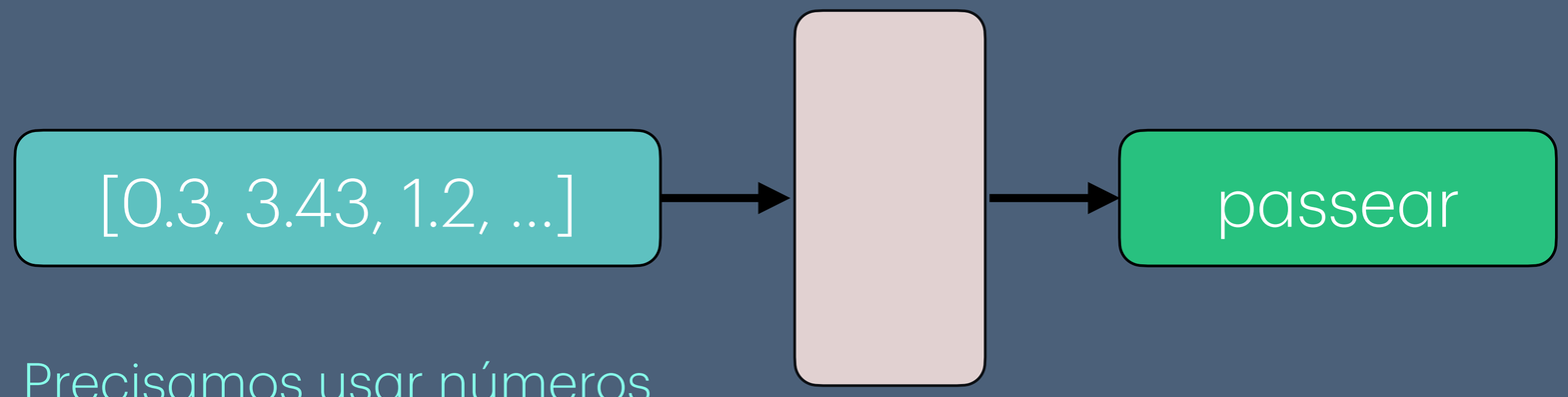
Como prever a próxima palavra?

Essa manhã eu levei meu cachorro para passear

considerando essas palavras

prever

Representando palavras para Redes Neurais



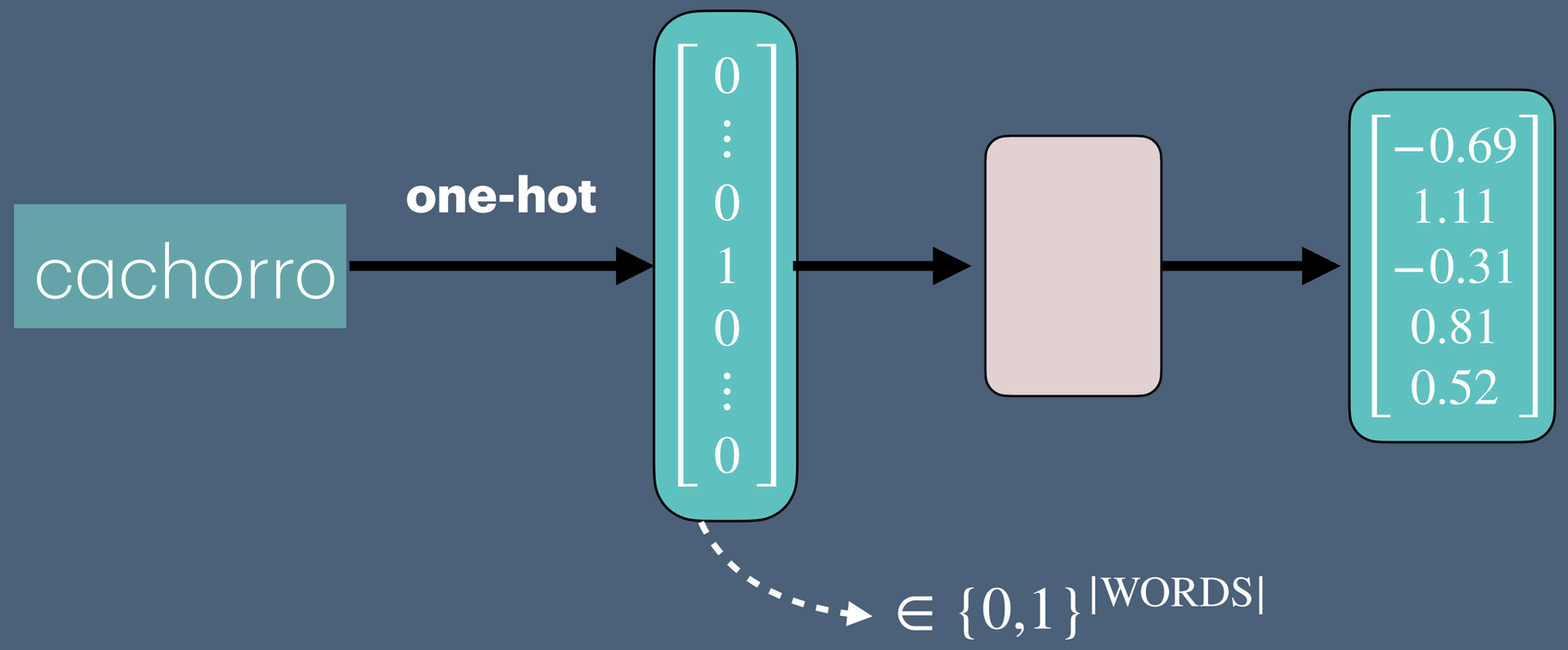
Precisamos usar números

Como representar palavras

Essa manhã eu levei meu cachorro para

Essa	manhã	eu	levei	meu	cachorro	para
$\begin{bmatrix} 0.82 \\ -0.15 \\ 1.04 \\ 0.33 \\ -0.41 \end{bmatrix}$	$\begin{bmatrix} 0.76 \\ -0.09 \\ 0.97 \\ 0.28 \\ -0.35 \end{bmatrix}$	$\begin{bmatrix} 0.71 \\ -0.18 \\ 1.12 \\ 0.21 \\ -0.29 \end{bmatrix}$	$\begin{bmatrix} -0.88 \\ 1.03 \\ -0.22 \\ 0.74 \\ 0.39 \end{bmatrix}$	$\begin{bmatrix} -0.79 \\ 0.95 \\ -0.16 \\ 0.68 \\ 0.44 \end{bmatrix}$	$\begin{bmatrix} -0.69 \\ 1.11 \\ -0.31 \\ 0.81 \\ 0.52 \end{bmatrix}$	$\begin{bmatrix} 0.14 \\ 0.62 \\ -0.87 \\ -0.33 \\ 0.91 \end{bmatrix}$

Como representar palavras



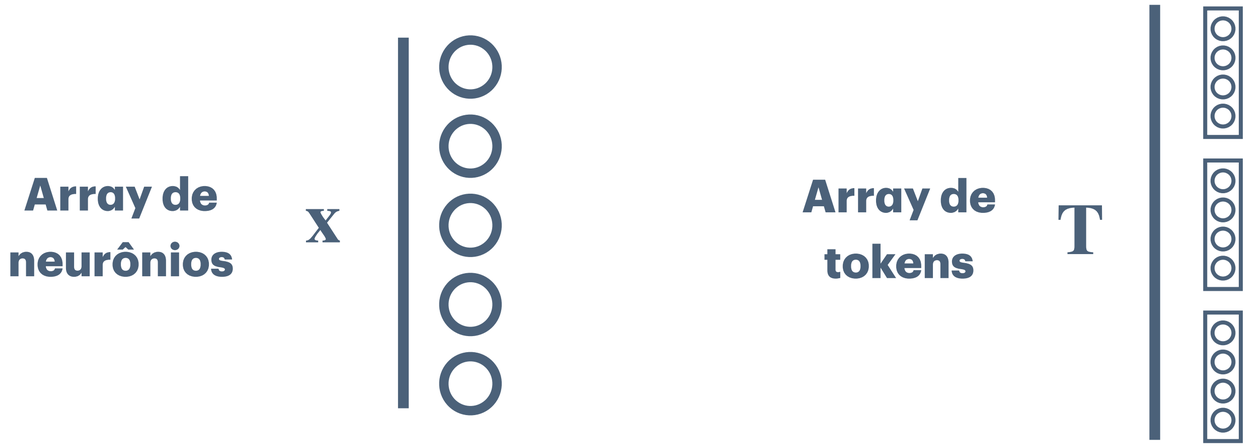
$\in \{0,1\}^{|\text{WORDS}|}$

$|\text{WORDS}| \approx 10^5$

Tokens

Uma nova forma de representar dados

- Um **token** é um vetor de números — uma unidade de informação
- Também podemos ver como um array de neurônios — Transformers operam em **arrays de tokens**.



Tokens

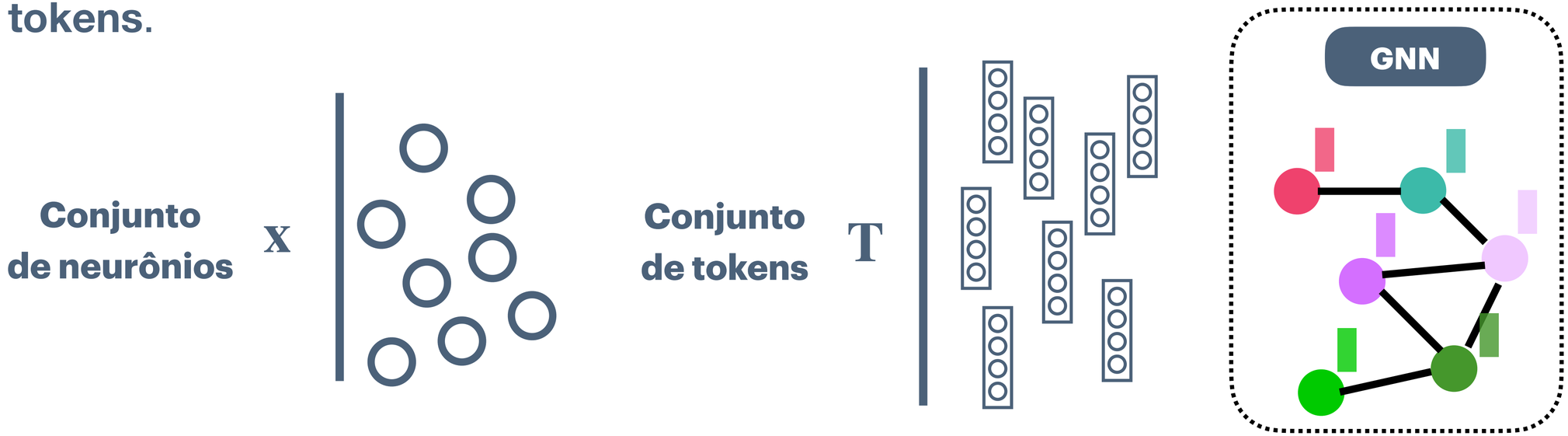
Uma nova forma de representar dados

- Um **token** é um vetor de números — uma unidade de informação
- Também podemos ver como um array de neurônios — Transformers operam em **arrays de tokens**.

Tokens

Uma nova forma de representar dados

- Um **token** é um vetor de números — uma unidade de informação
- Também podemos ver como um array de neurônios — Transformers operam em **arrays de tokens**.

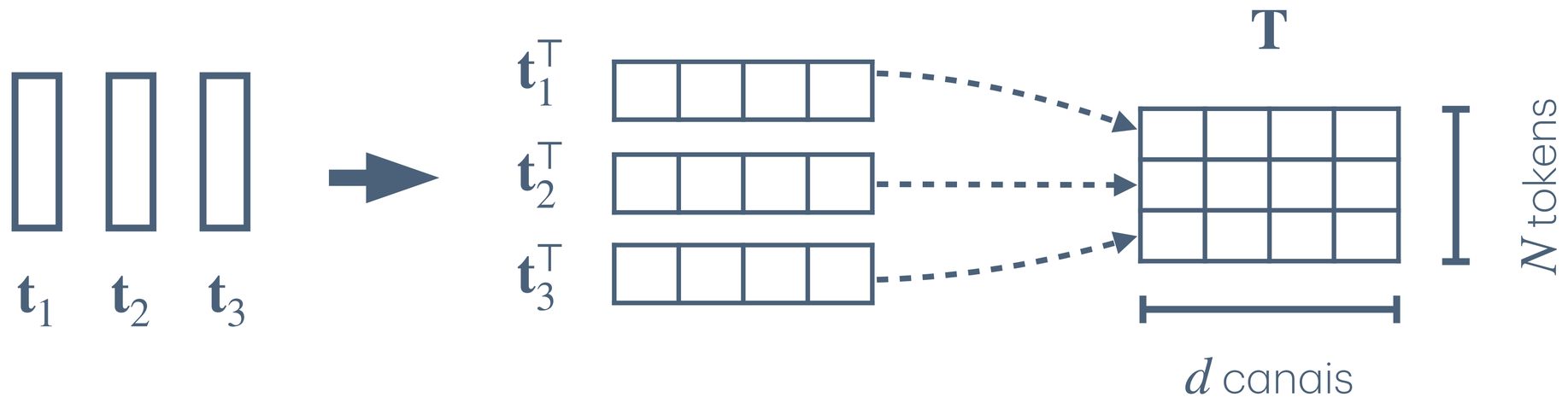


Tokenização

- O dado de entrada é quebrado em pedaços.
- Cada pedaço é representado por (transformado em) um token $t \in \mathbb{R}^d$.

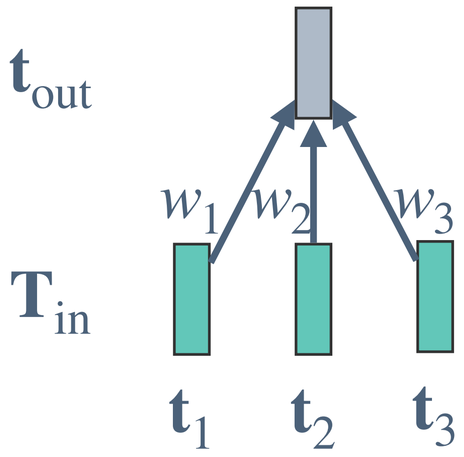
Operações em tokens

Notação:



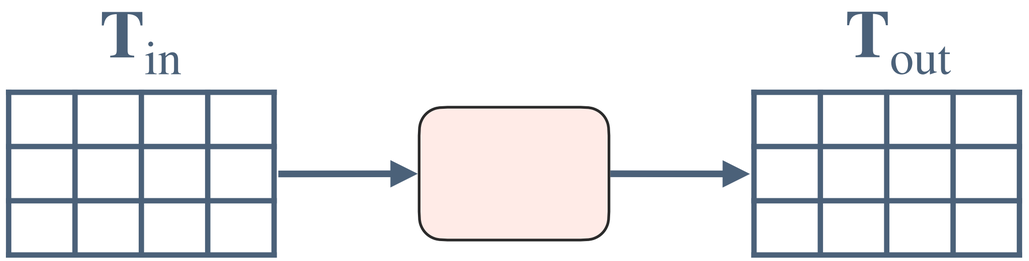
Operações em tokens

Combinações lineares de tokens



$$\mathbf{t}_{\text{out}} = w_1\,\mathbf{t}_1 + w_2\,\mathbf{t}_2 + w_3\,\mathbf{t}_3$$

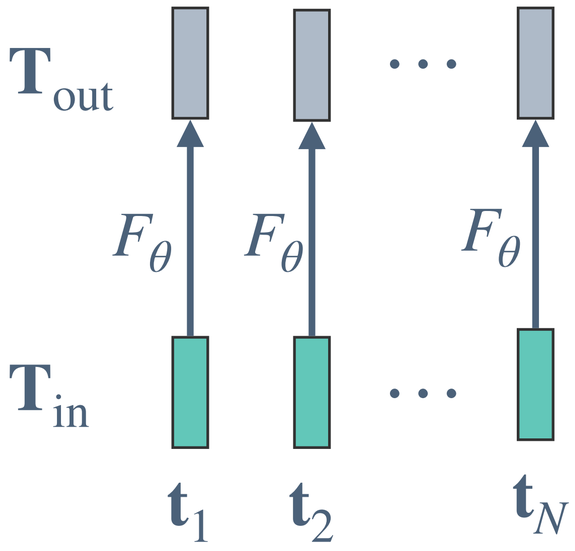
$$\mathbf{T}_{\text{out}}[i,\,:] = \sum_{j=1}^{N} w_{i,j}\mathbf{T}_{\text{in}}[j,\,:]$$



$$\mathbf{T}_{\text{out}} = \mathbf{W}\,\mathbf{T}_{\text{in}}$$

Operações em tokens

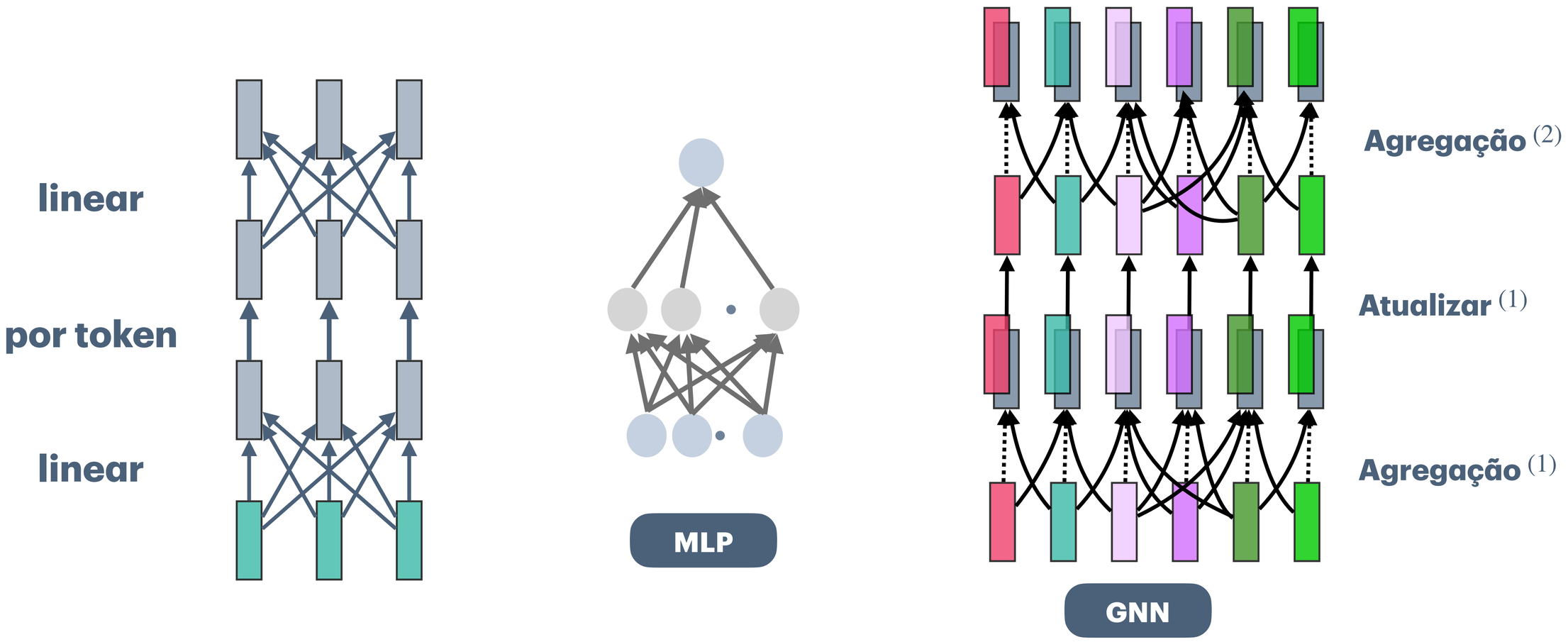
Operação token-wise



$$\mathbf{T}_{\text{out}} = \begin{bmatrix} F_\theta\left(\mathbf{T}_{\text{in}}[0, :]\right) \\ F_\theta\left(\mathbf{T}_{\text{in}}[1, :]\right) \\ \vdots \\ F_\theta\left(\mathbf{T}_{\text{in}}[N, :]\right) \end{bmatrix}$$

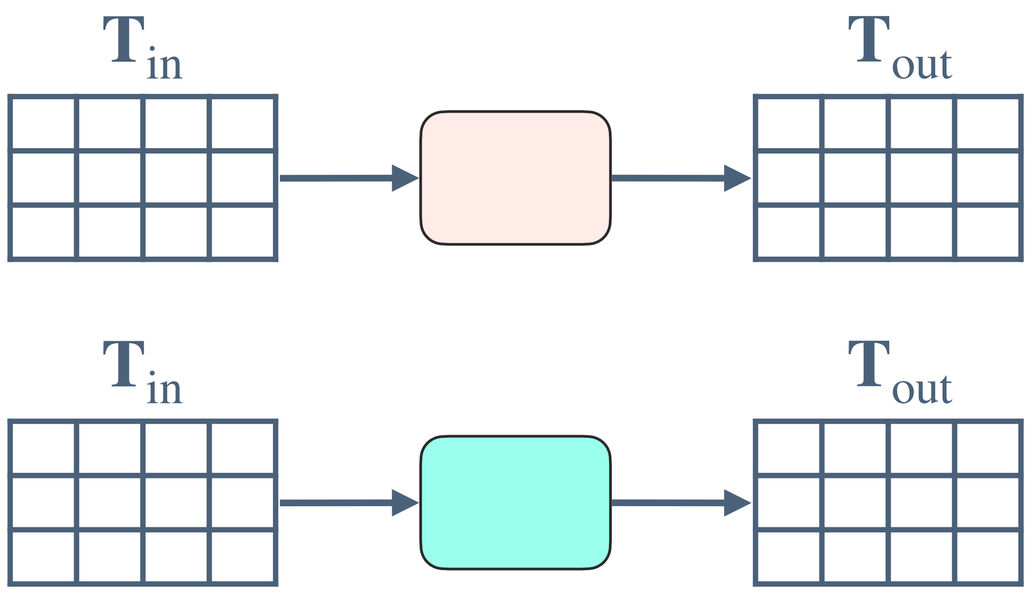
“não-linearidade em tokens”

Operações em tokens



Atenção

Atenção

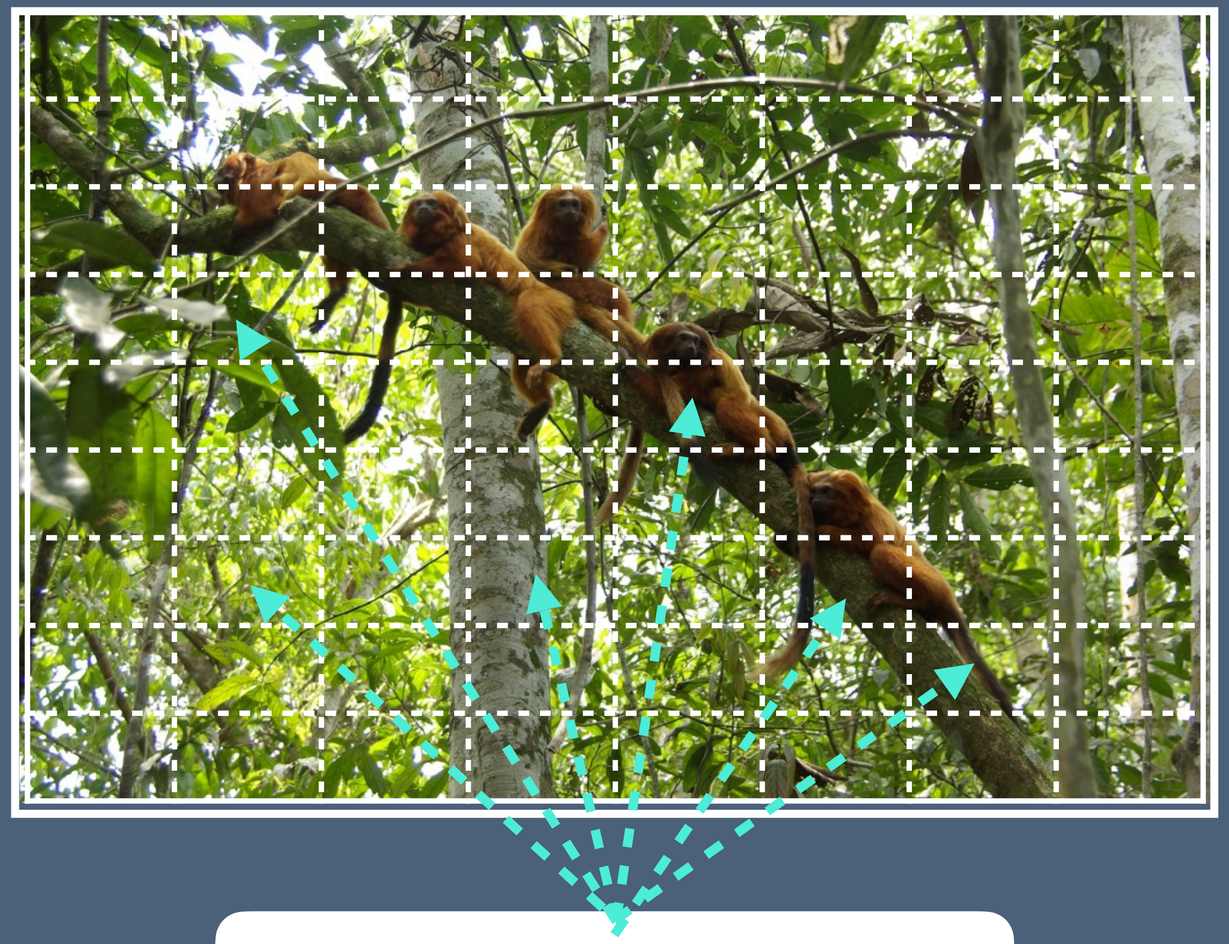


$$\mathbf{A} = f\,(\ldots)$$

$$\mathbf{T}_{\text{out}} = \mathbf{A}\,\mathbf{T}_{\text{in}}$$

$\mathbf{A}$ é função de algum dado de entrada. Esse dado nos diz quais tokens importam.

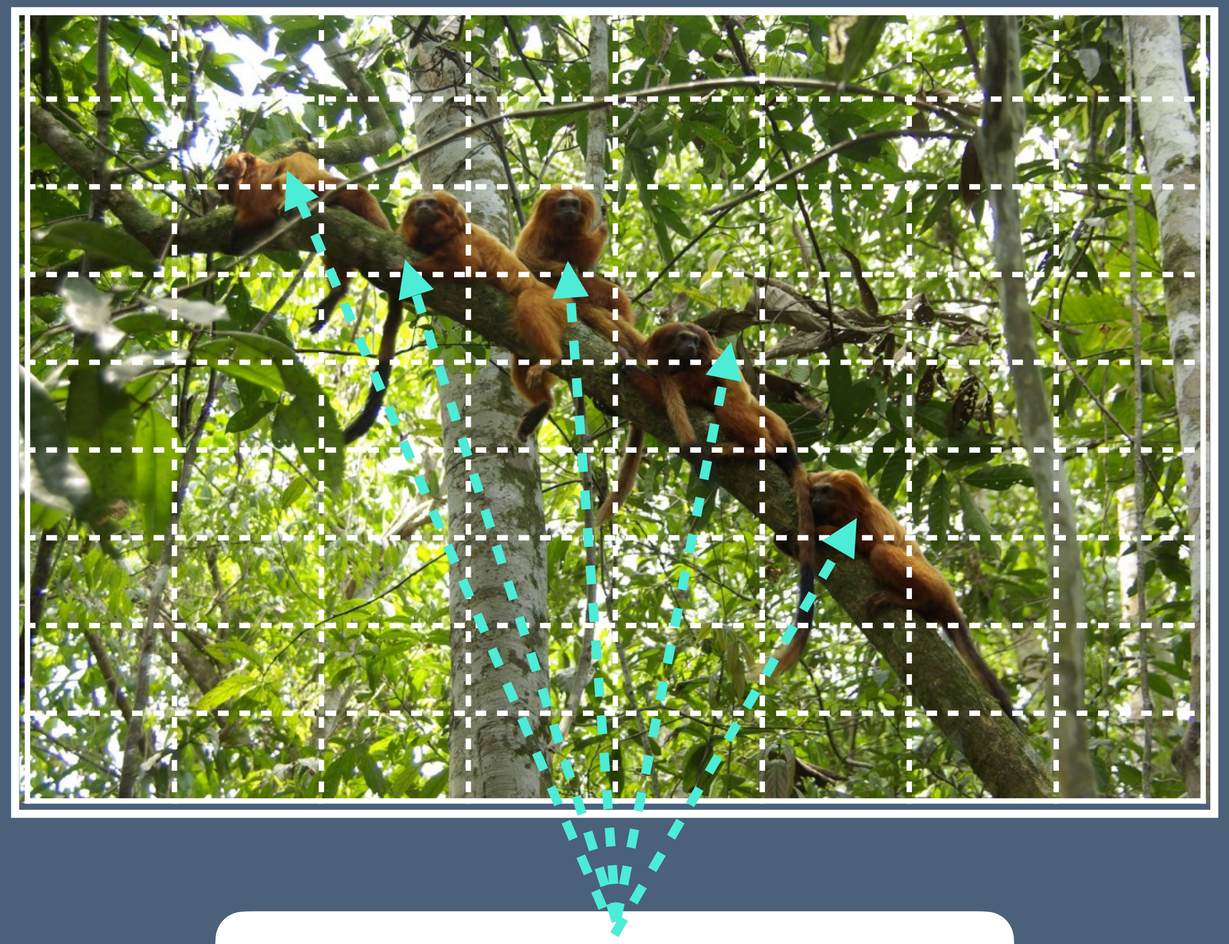
Atenção



$\mathbf{T}_{in}$

Quantos micos tem nessa imagem?

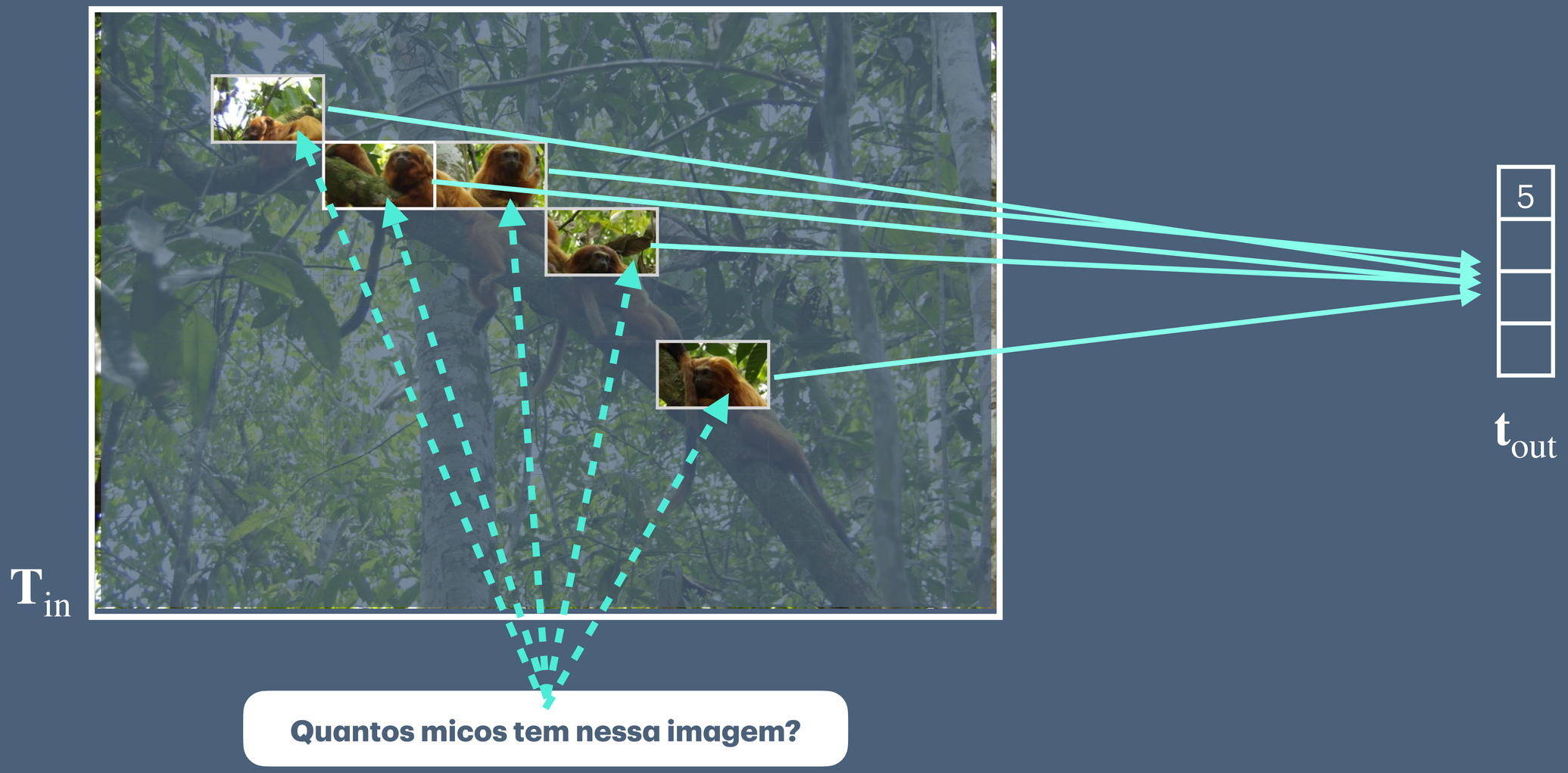
Atenção



$\mathbf{T}_{in}$

Quantos micos tem nessa imagem?

Atenção

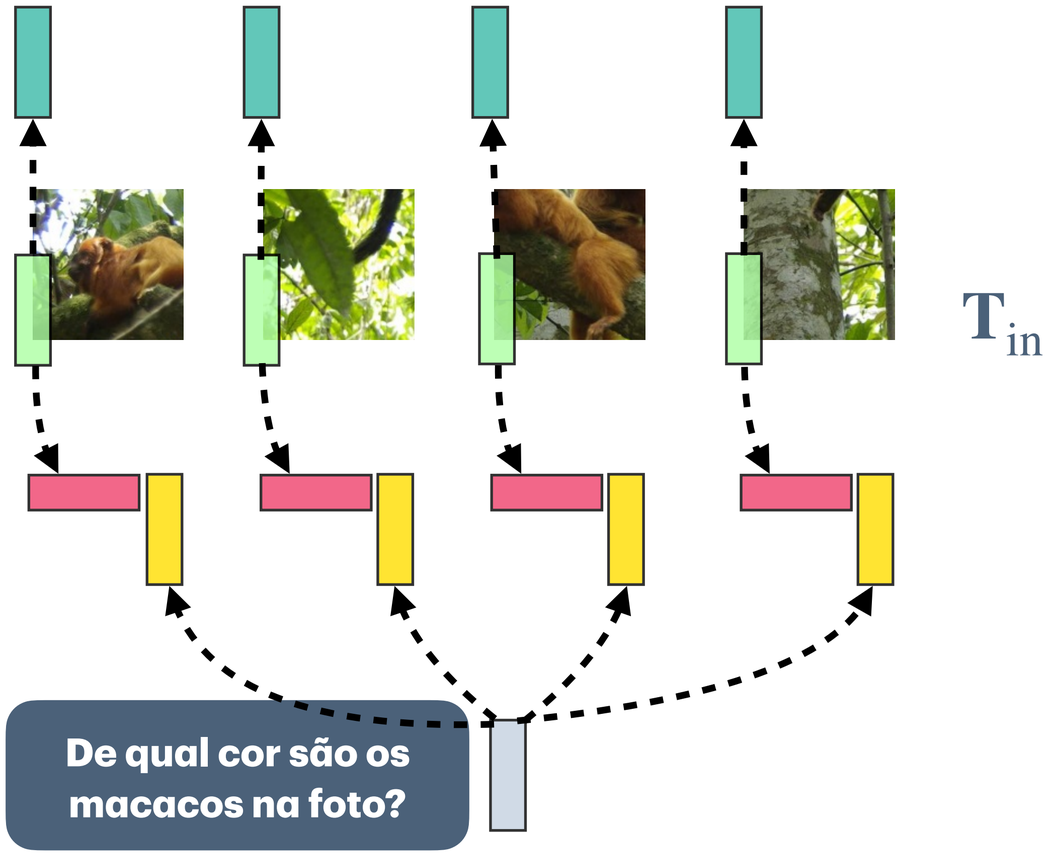


Atenção

Mecanismo de Atenção

- **Consulta — query** $\mathbf{q} = \mathbf{W}_q \mathbf{t}_{\text{consulta}}$
- **Chave — key** $\mathbf{k}_i = \mathbf{W}_k \mathbf{t}_i$
- **Valor — value** $\mathbf{v}_i = \mathbf{W}_v \mathbf{t}_i$

$$s = [q^\top k_1, q^\top k_2, \ldots, q^\top k_N]$$

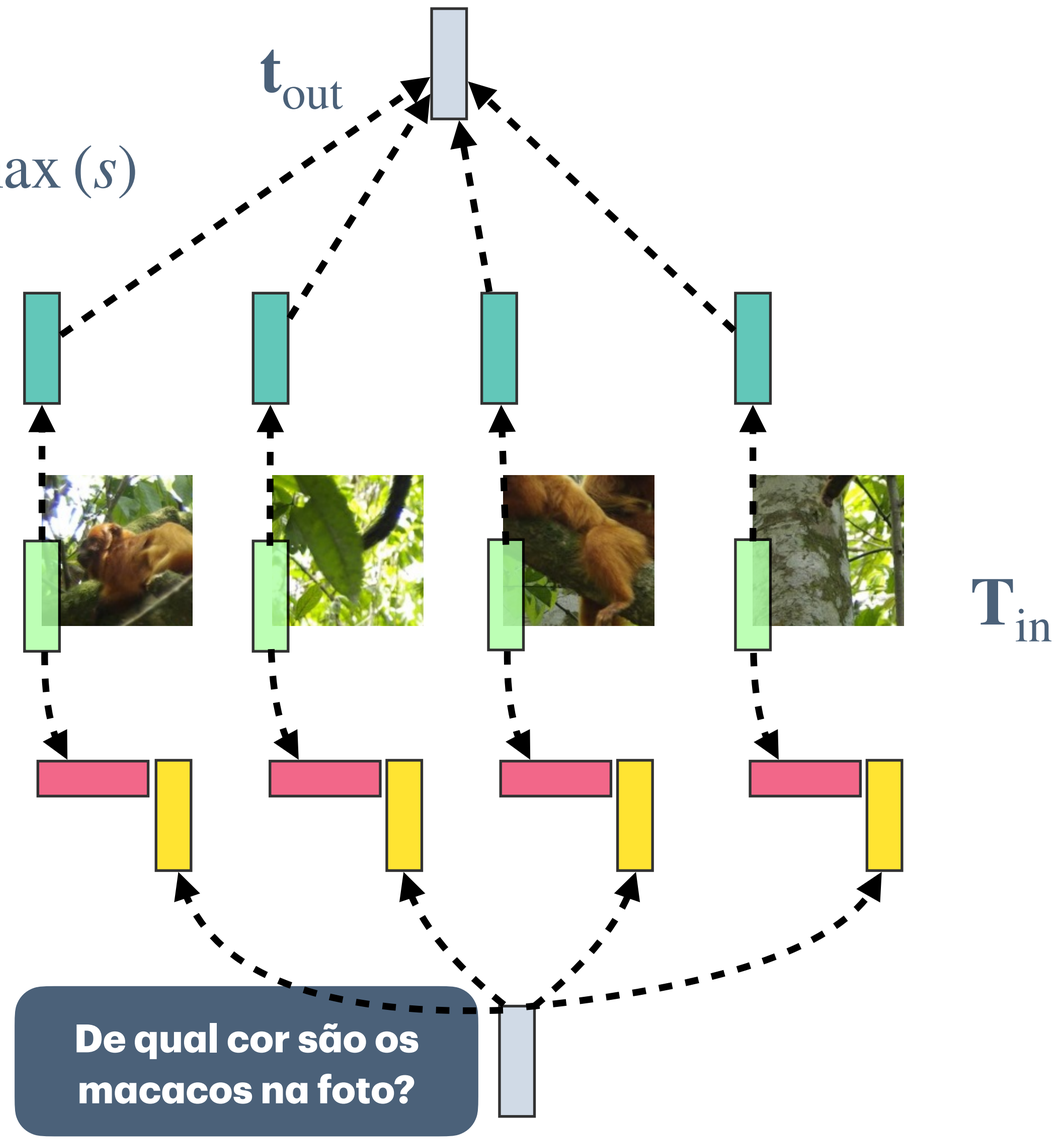


Mecanismo de Atenção

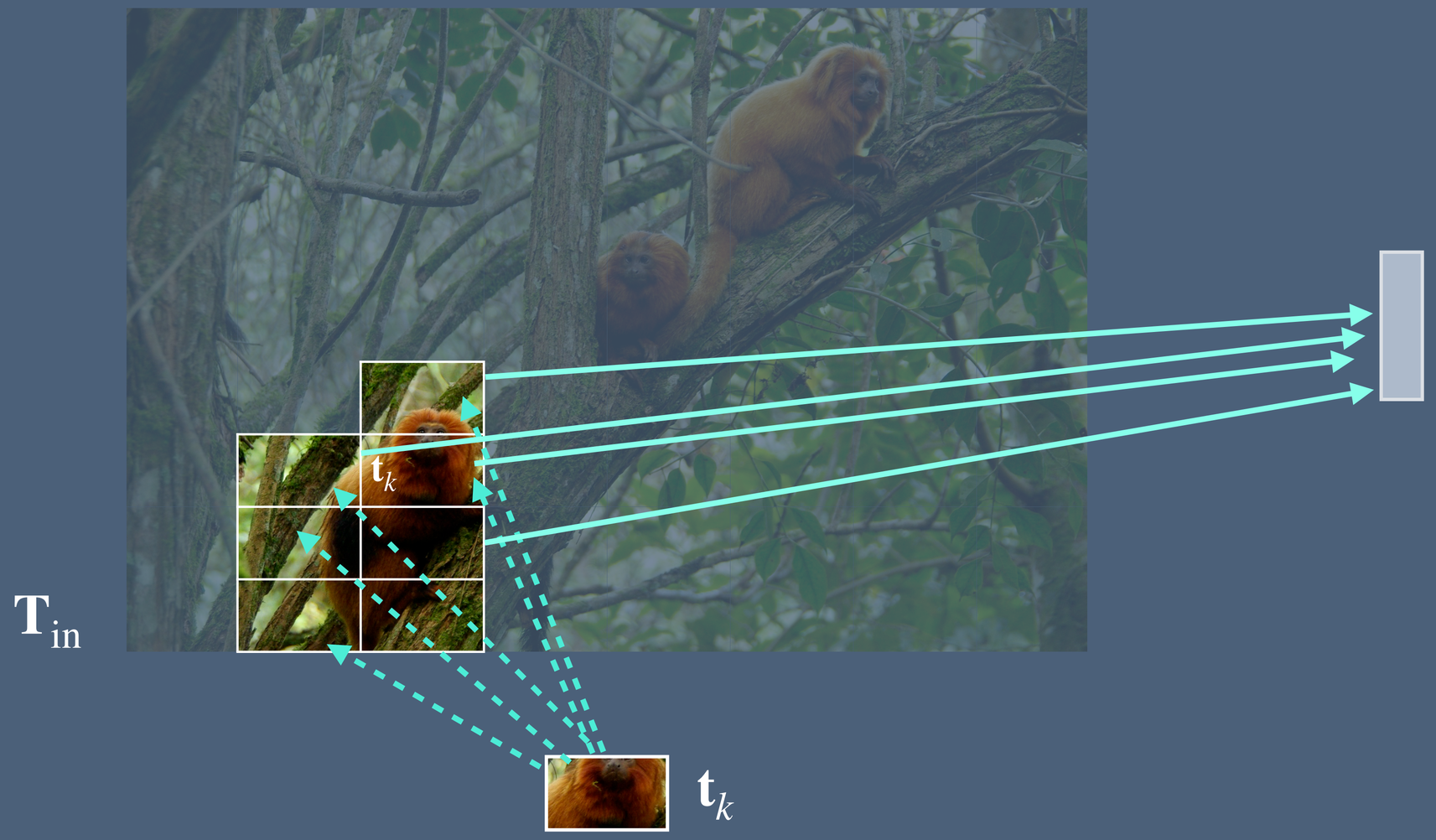
$$\mathbf{A} = \text{softmax}\,(s)$$

- **Consulta — query** $\mathbf{q} = \mathbf{W}_q\,\mathbf{t}_{\text{consulta}}$
- **Chave — key** $\mathbf{k}_i = \mathbf{W}_k\,\mathbf{t}_i$
- **Valor — value** $\mathbf{v}_i = \mathbf{W}_v\,\mathbf{t}_i$

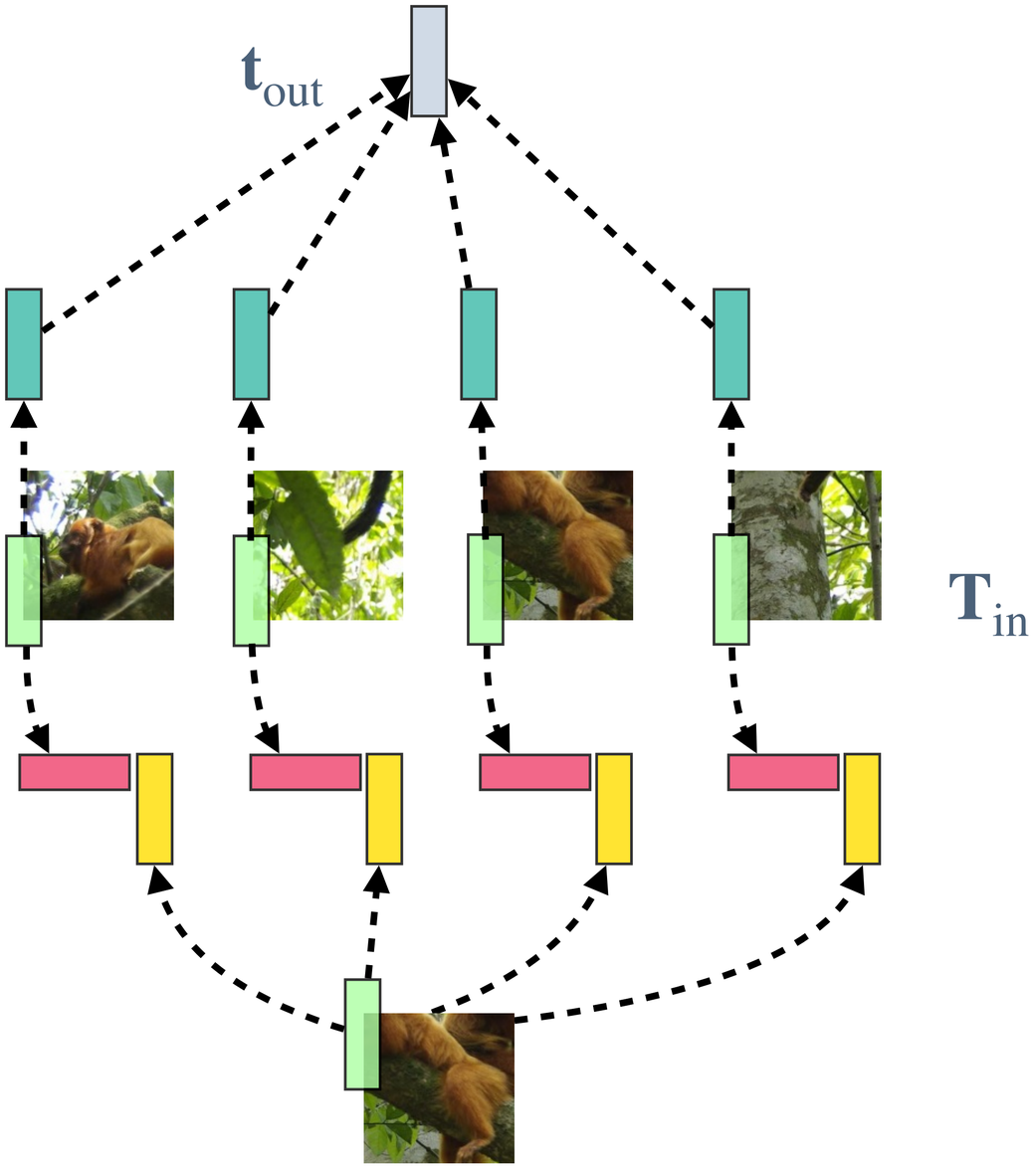
$$s = \left[q^\top k_1, q^\top k_2, \ldots, q^\top k_N\right]$$



Auto-Atenção



Auto-Atenção



Emerging Properties in Self-Supervised Vision Transformers — Caron et al 2021

Auto-Atenção

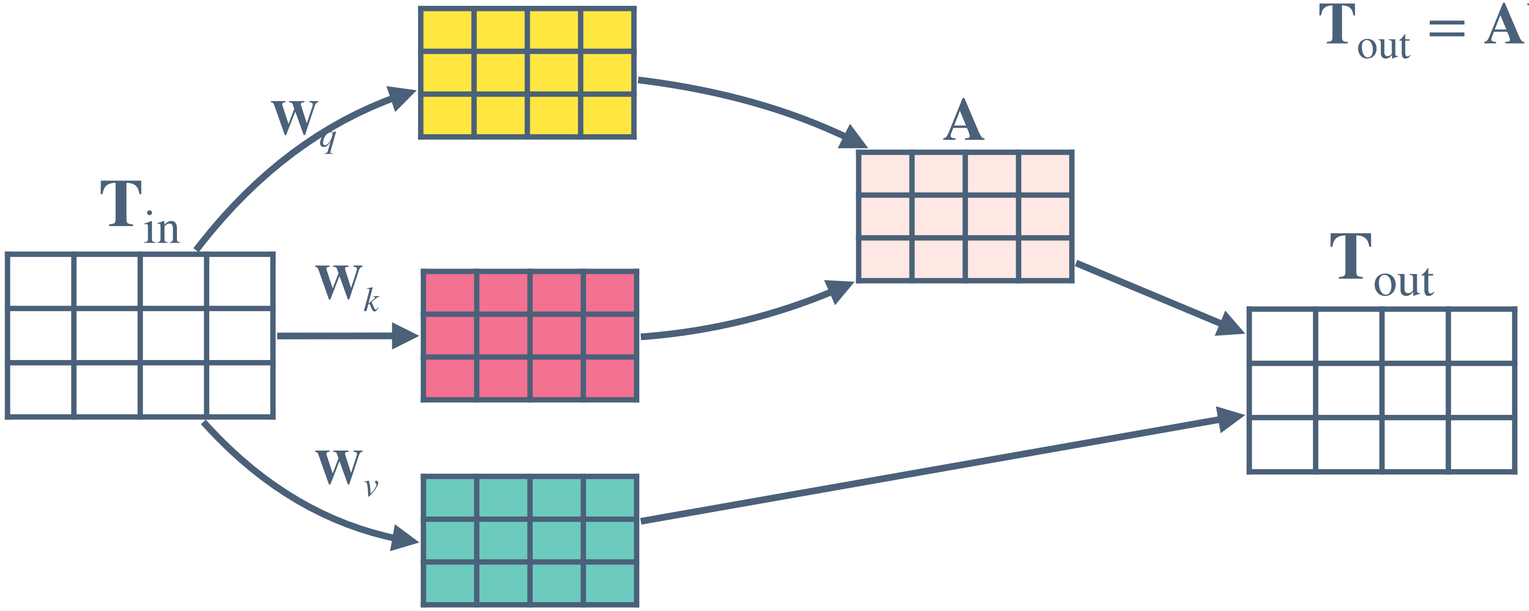
$$\mathbf{Q}_{\text{in}} = \mathbf{T}_{\text{in}}\mathbf{W}_q^\top$$

$$\mathbf{K}_{\text{in}} = \mathbf{T}_{\text{in}}\mathbf{W}_k^\top$$

$$\mathbf{V}_{\text{in}} = \mathbf{T}_{\text{in}}\mathbf{W}_v^\top$$

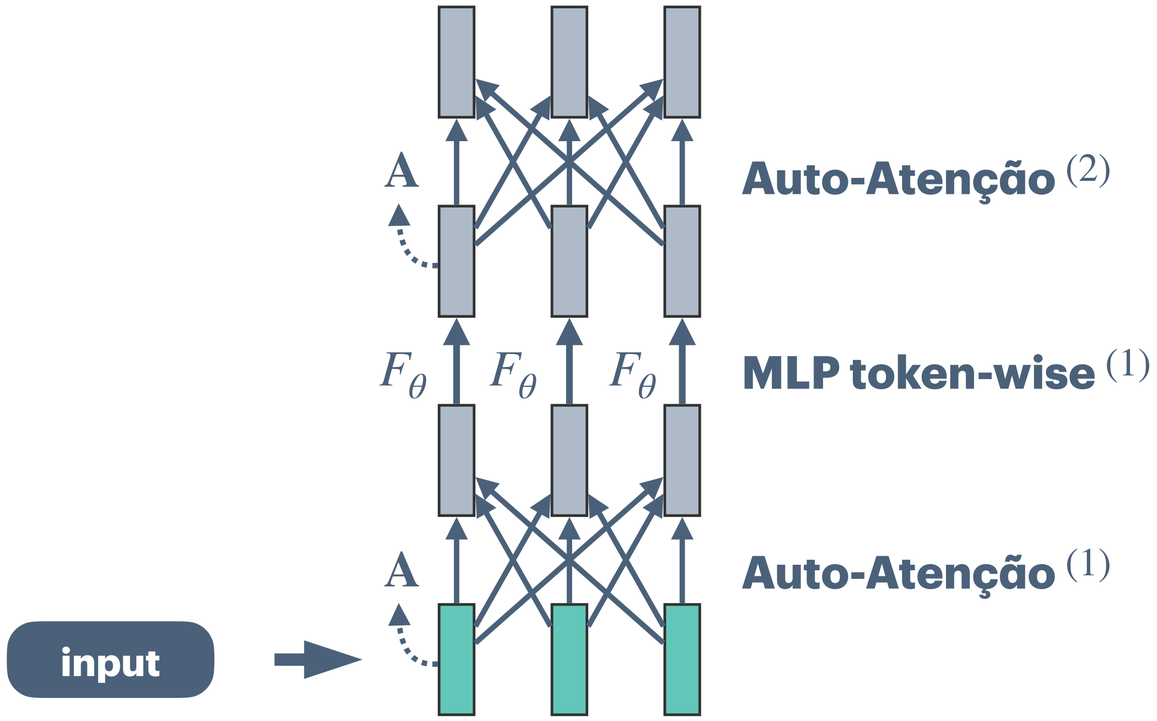
$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}_{\text{in}}\mathbf{K}_{\text{in}}^\top}{\sqrt{m}}\right)$$

$$\mathbf{T}_{\text{out}} = \mathbf{A}\mathbf{V}_{\text{in}}$$



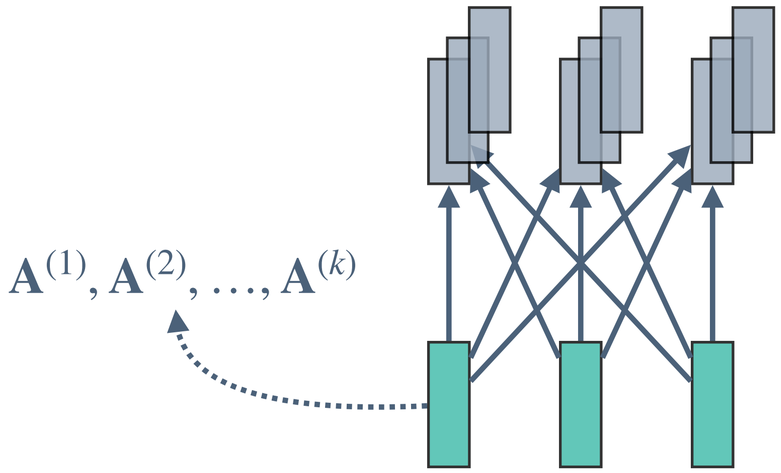
Tranformer

Arquitetura do Transformer:



Tranformer

Auto-atenção com múltiplas cabeças (MSA)



$$\mathbf{T}_{\text{out}}^{(i)} = \text{attn}^{(i)}\left(\mathbf{T}_{\text{in}}\right), \text{ para } i \in \{1,2,\ldots,k\}$$

$$\bar{\mathbf{T}}_{\text{out}} = \begin{bmatrix} \mathbf{T}_{\text{out}}^{(1)}[0,\ :] & \ldots & \mathbf{T}_{\text{out}}^{(k)}[0,\ :] \\ \vdots & \vdots & \vdots \\ \mathbf{T}_{\text{out}}^{(1)}[N-1,\ :] & \ldots & \mathbf{T}_{\text{out}}^{(k)}[N-1,\ :] \end{bmatrix}_{N \times kv}$$

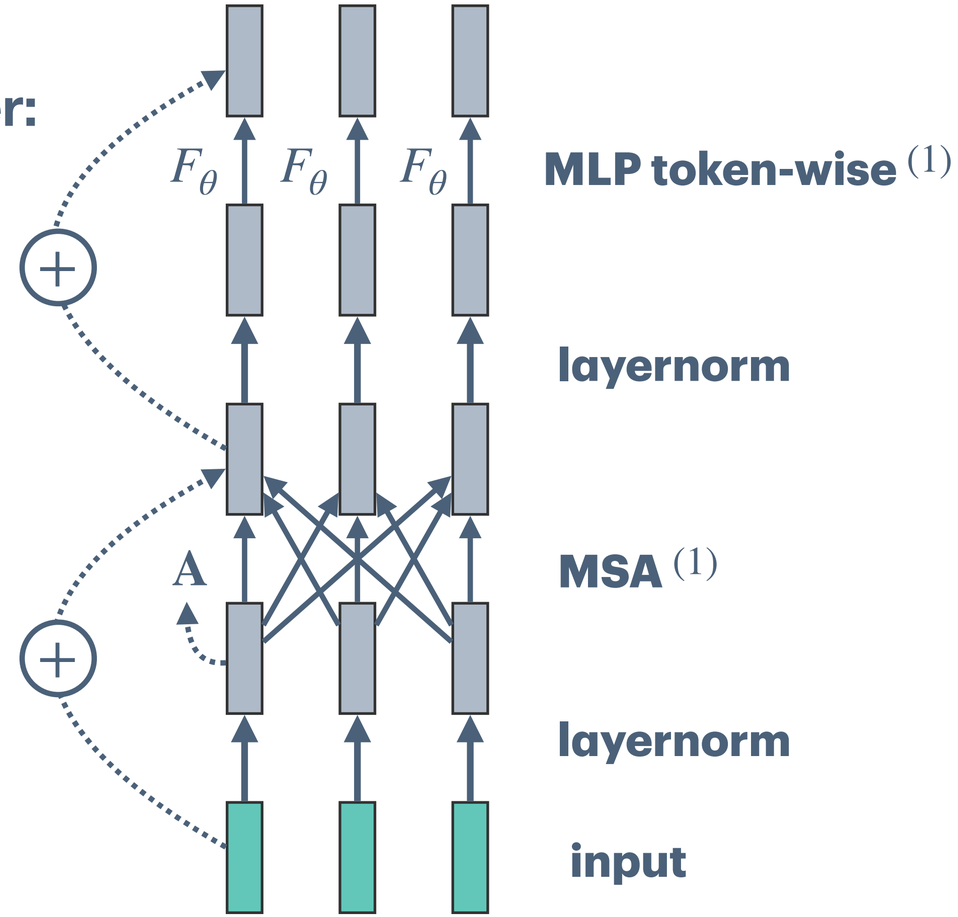
$$\mathbf{T}_{\text{out}} = \bar{\mathbf{T}}_{\text{out}} \mathbf{W}_{\text{MSA}} \qquad \mathbf{W}_{\text{MSA}} \in \mathbb{R}^{kv \times d}$$

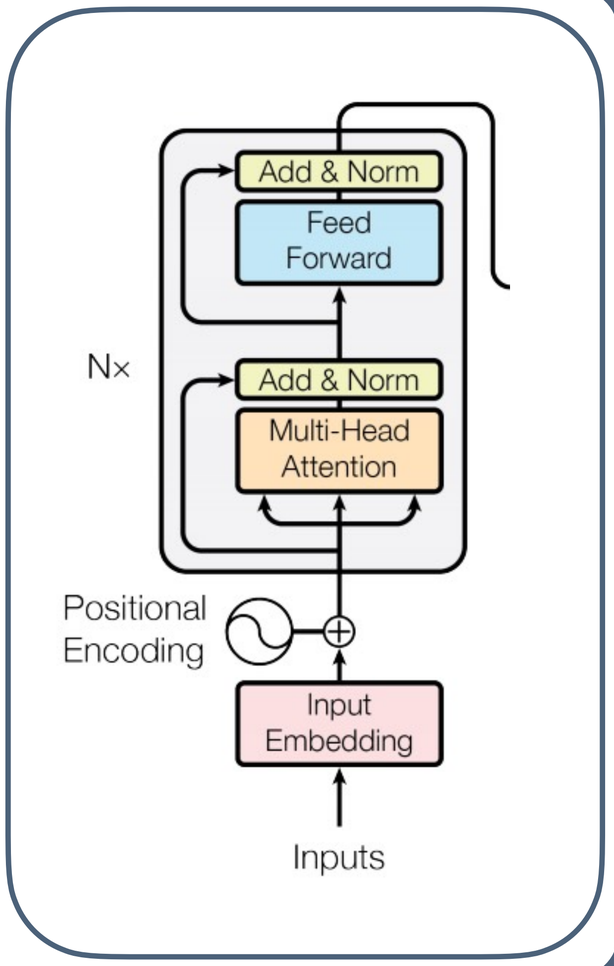
Tranformer

Arquitetura do Transformer:

layernorm:

$$x_{\text{out}}[k] = \frac{x_{\text{in}}[k] - \mathbb{E}[x_{\text{in}}]}{\sqrt{\text{Var}(\text{x}_{\text{in}})}}$$

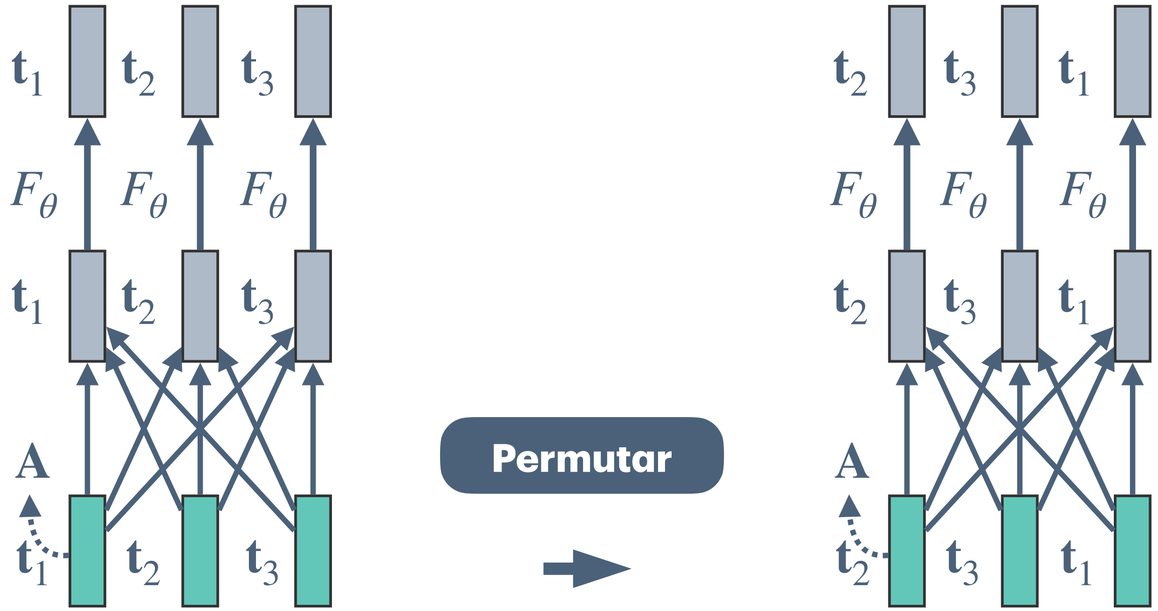




Encoder Posicional

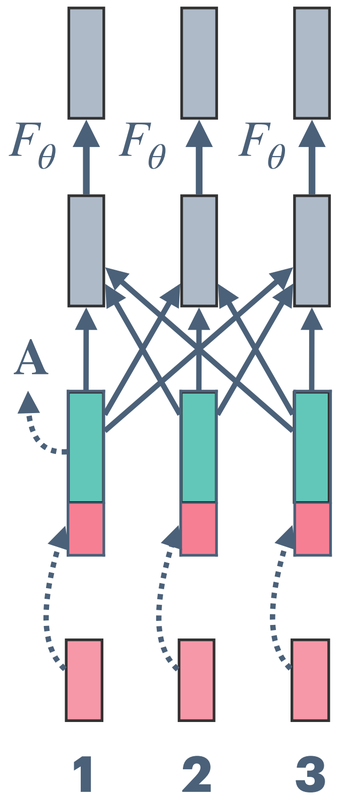
Tranformer = Set2Set

Transformers são equivariantes por permutação



Encoder Posicional

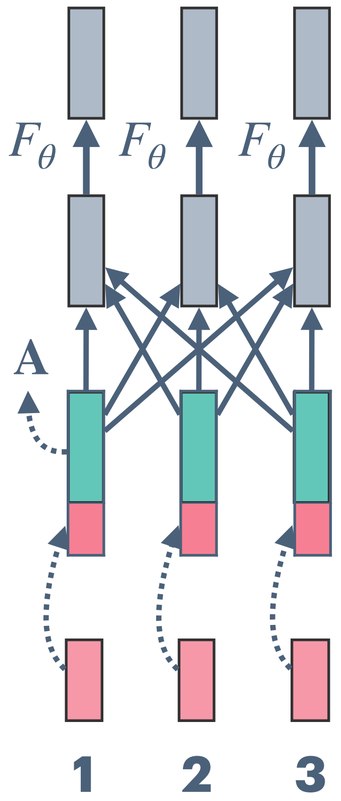
- **Cada índice ou posição é codificado em um vetor**
- **Adicionamos o vetor com a informação da posição em cada um dos tokens associados.**



Encoder Posicional

O que queremos que eles satisfaçam?

- Queremos que a distância entre os vetores reflita a diferença das posições associadas
- Vetores com valores entre 0 e 1.
- Invariantes pelo tamanho da sequência.
- Determinísticos.



Encoder Posicional

Exemplo: Encoder por frequências (do “Attention is all you need”)

$$p_{k,2i} = \sin\left(\frac{k}{n^{2i/d}}\right) \qquad p_{k,2i+1} = \cos\left(\frac{k}{n^{2i/d}}\right)$$

- $p_{k,j}$ — entrada j do encode para posição k
- d — dimensão do encode
- n — parâmetro de simetria (e.g. = 10.000)

Encoder Posicional

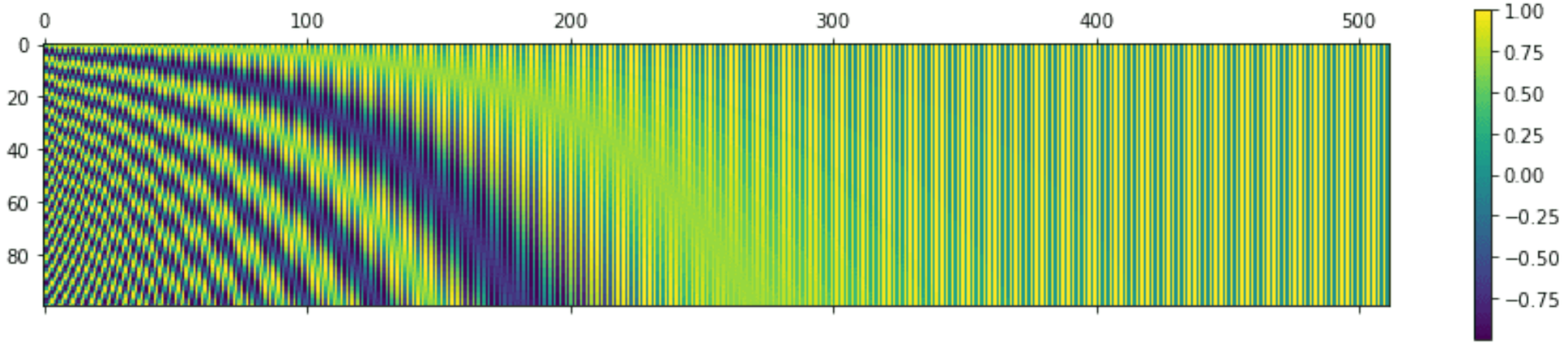
Exemplo: Encoder por frequências (do “Attention is all you need”)

$$p_{k,2i} = \sin\left(\frac{k}{n^{2i/d}}\right) \qquad p_{k,2i+1} = \cos\left(\frac{k}{n^{2i/d}}\right)$$

n = 10000

d = 512

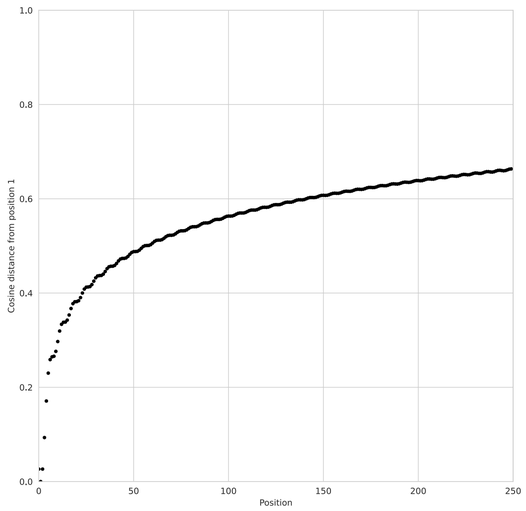
Seq. Length = 100

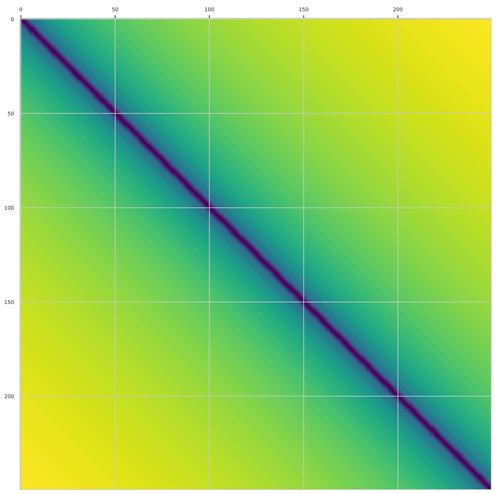


Encoder Posicional

Exemplo: Encoder por frequências (do “Attention is all you need”)

$$p_{k,2i} = \sin\left(\frac{k}{n^{2i/d}}\right) \qquad p_{k,2i+1} = \cos\left(\frac{k}{n^{2i/d}}\right)$$

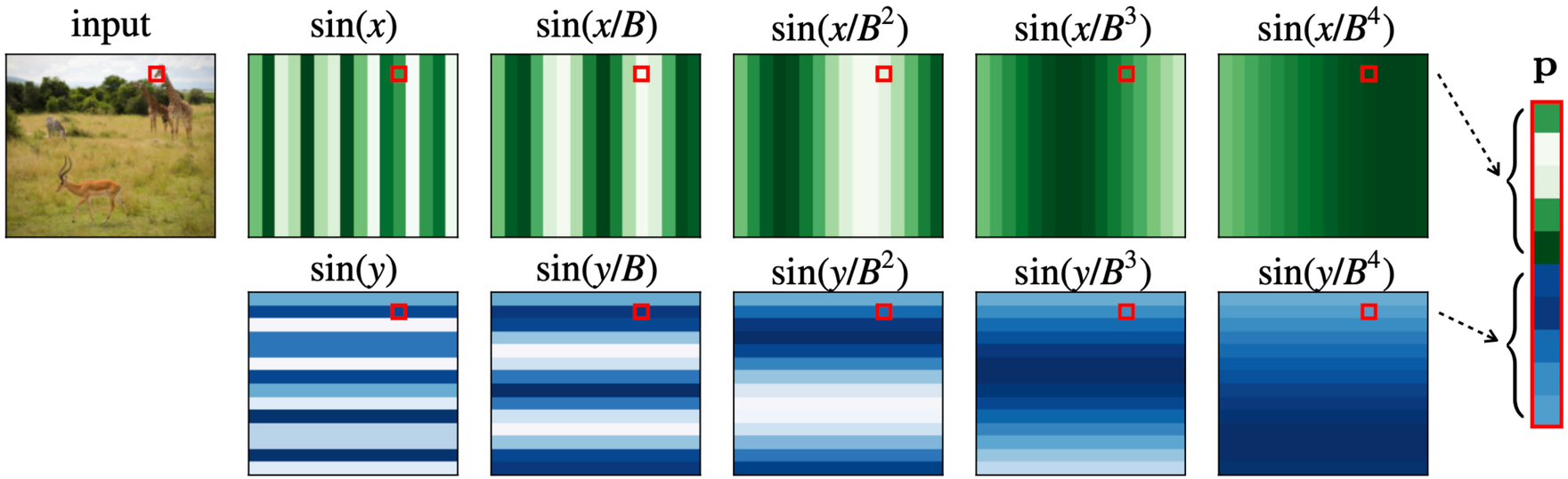


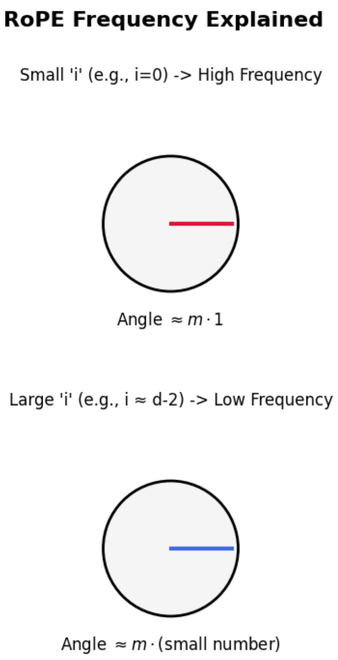
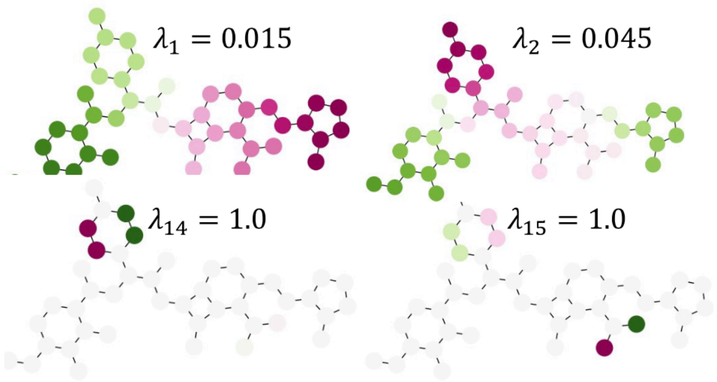
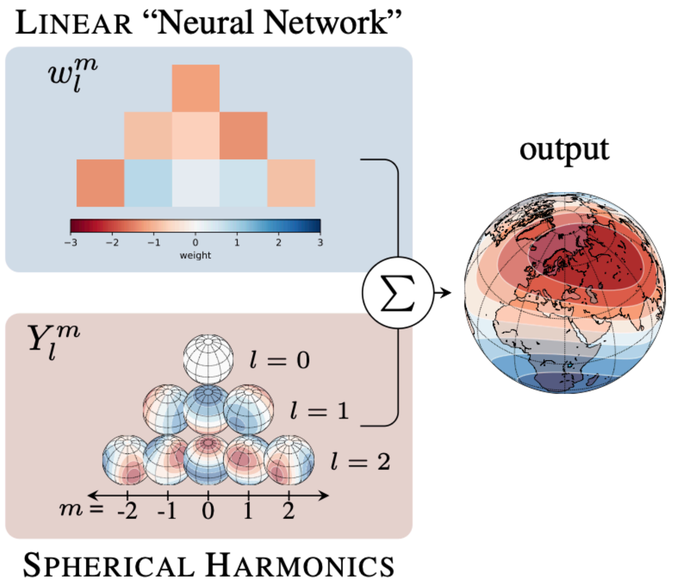


$$\cos(\theta) = \frac{x^\top y}{\|x\|\,\|y\|}$$

$$d_{\cos}(x, y) = 1 - \frac{x^\top y}{\|x\|\,\|y\|}$$

Encoder Posicional



Encoder Posicional

Outros encoders: